INTERNAL API + MOBILE

Penetration Test Report

Prepared for **Noriga Payments API**

ENGAGEMENT	NRX-26Q1	WINDOW	Feb 9, 2026 – Apr 17, 2026
VERSION	1.0	ISSUED	May 4, 2026
LEAD	Rami Moreau	TEAM	Kenji Oduya, Rami Moreau, Mira Vos
CLASSIFICATION	Confidential — Restricted		

Document Control

Revision History

Version	Date	Author	Description
1.0	May 4, 2026	Rami Moreau	Initial release

Confidentiality Notice

This document contains confidential and proprietary information belonging to the client and the assessing firm. It is intended solely for the use of the named recipient(s). Disclosure, copying, distribution, or use of the contents of this document by any person other than the intended recipient is strictly prohibited.

The findings, recommendations, and opinions expressed in this report reflect the state of the assessed systems at the time of testing only. The assessing firm accepts no liability for the use or misuse of information contained herein.

> **Distribution:** This report is classified *Confidential — Restricted*. Retain securely and do not transmit over unencrypted channels. Evidence and raw data are retained for 90 days and then cryptographically shredded.

Contents

Executive Summary

A total of 2 vulnerabilities were identified during the Internal API + Mobile assessment of Noriga Payments API. 1 critical or high-severity finding requires immediate remediation.

Findings at a Glance

All issues identified during the engagement are summarized in **Table 1**.

Critical	High	Medium	Low	Informational
0	1	1	0	0

Table 1. *Total findings by severity.*

Project Scope

The following assets were in scope for this engagement:

- `api.noriga.internal`
- `iOS v4.8`
- `Android v4.8`
- `nothing.com`

Key Security Strengths

The environment demonstrated baseline security controls. Standard authentication mechanisms and input validation were observed on primary endpoints.

Key Areas for Improvement

- [HIGH] **Hardcoded API key in mobile app binary** — Remove hardcoded secrets. Use certificate pinning and runtime key fetching with proper authenticatio
- [MEDIUM] **Broken object level authorization on payment endpoint** — Implement BOLA/IDOR protection by always verifying that the authenticated user owns the requested re

Strategic Recommendations

Immediate Actions (0–30 Days)

The following critical and high-severity findings require immediate attention:

- HIGH **F-P2-002** — Hardcoded API key in mobile app binary: Remove hardcoded secrets. Use certificate pinning and runtime key fetching with

Short-Term Improvements (30–90 Days)

Address medium-severity findings to reduce overall attack surface:

- MEDIUM **F-P2-001** — Broken object level authorization on payment endpoint

Long-Term Security Hardening

The following strategic initiatives are recommended to strengthen the overall security programme:

- Implement a continuous vulnerability management programme with quarterly assessments.
- Establish a formal secure development lifecycle (SDLC) with security gates at design, code review, and deployment stages.
- Deploy a security information and event management (SIEM) solution for real-time threat detection.
- Conduct annual penetration tests against all internet-facing assets and internal network segments.

Technical Details

Technical Scope

The following assets were evaluated during this engagement:

Asset / URL	Type	Notes
`api.noriga.internal`	—	—
`iOS v4.8`	—	—
`Android v4.8`	—	—
`nothing.com`	—	—

Testing Methodology

The engagement followed the *PTES* execution phases mapped to *OWASP WSTG v4.2* controls, combining authenticated and unauthenticated test runs across the full assessment window.

#	Activity	Primary Tooling
01	Reconnaissance	OSINT, DNS enumeration, asset discovery, Shodan
02	Threat Modelling	STRIDE analysis against service map
03	Vulnerability Analysis	Burp Suite Pro, Semgrep, Nuclei, manual testing
04	Exploitation	Controlled PoC — scoped to confirm impact only
05	Post-Exploitation	Read-only confirmation within agreed rules of engagement
06	Reporting	CVSS 3.1 scoring, CWE/OWASP mapping

Engagement Timeline

Milestone	Date / Period
Assessment Start	`Feb 9, 2026`
Assessment End	`Apr 17, 2026`
Report Issued	`May 4, 2026`
Retest Window	`90 days from report issue`

> **Rules of Engagement —** No destructive testing, no customer PII handling, and no testing of out-of-scope third-party processors. Testing was conducted during agreed windows only.

Detailed Findings

All 2 identified findings are enumerated below, ordered by severity. Full technical details follow on subsequent pages.

ID	Severity	Title	Component	CVSS
F-P2-002	HIGH	Hardcoded API key in mobile app binary	—	5.3
F-P2-001	MEDIUM	Broken object level authorization on payment endpoint	—	5.9

Refer to Appendix A for CVSS severity rating definitions. Each finding on the following pages includes full technical details, reproduction steps, impact analysis, and remediation guidance.

High Risk Findings

HIGH **1 finding** Avg CVSS: 5.3

High-severity findings pose significant risk to system integrity, confidentiality, or availability. Remediate within 30 days.

ID	Title	CVSS	Page
F-P2-002	Hardcoded API key in mobile app binary	5.3	9

FINDING **F-P2-002**

Hardcoded API key in mobile app binary

SEVERITY	HIGH
CVSS V3.1	**5.3** `AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N`
CWE	`CWE-798`
OWASP	`A02:2021`
AFFECTED	`iOS v4.8`

Description

Reverse engineering the mobile app binary reveals hardcoded API keys used to authenticate with the backend.

Affected Scope

- `iOS v4.8`
- `Android v4.8`

Impact

Unauthenticated access to backend APIs. Key cannot be easily rotated without app update.

Recommendations

Remove hardcoded secrets. Use certificate pinning and runtime key fetching with proper authentication.

Technical Details

1. Download the app APK/IPA
2. Decompile using jadx/frida
3. Search for API key patterns
4. Use extracted key to make unauthenticated API calls
5. Download the app APK/IPA
6. Decompile using jadx/frida
7. Search for API key patterns
8. Use extracted key to make unauthenticated API calls
9. Download the app APK/IPA
10. Decompile using jadx/frida
11. Search for API key patterns
12. Use extracted key to make unauthenticated API calls
13. Download the app APK/IPA
14. Decompile using jadx/frida
15. Search for API key patterns
16. Use extracted key to make unauthenticated API calls

17. Download the app APK/IPA
18. Decompile using jadx/frida
19. Search for API key patterns
20. Use extracted key to make unauthenticated API calls
21. Download the app APK/IPA
22. Decompile using jadx/frida
23. Search for API key patterns
24. Use extracted key to make unauthenticated API calls
25. Download the app APK/IPA
26. Decompile using jadx/frida
27. Search for API key patterns
28. Use extracted key to make unauthenticated API calls
29. Download the app APK/IPA
30. Decompile using jadx/frida
31. Search for API key patterns
32. Use extracted key to make unauthenticated API callsa
33. Download the app APK/IPA
34. Decompile using jadx/frida
35. Search for API key patterns
36. Use extracted key to make unauthenticated API calls
37. Download the app APK/IPA
38. Decompile using jadx/frida
39. Search for API key patterns
40. Use extracted key to make unauthenticated API calls
41. Download the app APK/IPA
42. Decompile using jadx/frida
43. Search for API key patterns
44. Use extracted key to make unauthenticated API calls
45. Download the app APK/IPA
46. Decompile using jadx/frida
47. Search for API key patterns
48. Use extracted key to make unauthenticated API calls
49. Download the app APK/IPA
50. Decompile using jadx/frida
51. Search for API key patterns
52. Use extracted key to make unauthenticated API calls
53. Download the app APK/IPA
54. Decompile using jadx/frida

FINDING **F-P2-002** *CONTINUED*

57. Download the app APK/IPA
58. Decompile using jadx/frida
59. Search for API key patterns
60. Use extracted key to make unauthenticated API calls
61. Download the app APK/IPA
62. Decompile using jadx/frida
63. Search for API key patterns
64. Use extracted key to make unauthenticated API callsa
65. Download the app APK/IPA
66. Decompile using jadx/frida
67. Search for API key patterns
68. Use extracted key to make unauthenticated API calls
69. Download the app APK/IPA
70. Decompile using jadx/frida
71. Search for API key patterns
72. Use extracted key to make unauthenticated API calls
73. Download the app APK/IPA
74. Decompile using jadx/frida
75. Search for API key patterns
76. Use extracted key to make unauthenticated API calls
77. Download the app APK/IPA
78. Decompile using jadx/frida
79. Search for API key patterns
80. Use extracted key to make unauthenticated API calls
81. Download the app APK/IPA
82. Decompile using jadx/frida
83. Search for API key patterns
84. Use extracted key to make unauthenticated API calls
85. Download the app APK/IPA
86. Decompile using jadx/frida
87. Search for API key patterns
88. Use extracted key to make unauthenticated API calls
89. Download the app APK/IPA
90. Decompile using jadx/frida
91. Search for API key patterns
92. Use extracted key to make unauthenticated API calls
93. Download the app APK/IPA
94. Decompile using jadx/frida
95. Search for API key patterns

FINDING **F-P2-002** *CONTINUED*

98. Decompile using jadx/frida
99. Search for API key patterns
100. Use extracted key to make unauthenticated API calls
101. Download the app APK/IPA
102. Decompile using jadx/frida
103. Search for API key patterns
104. Use extracted key to make unauthenticated API calls
105. Download the app APK/IPA
106. Decompile using jadx/frida
107. Search for API key patterns
108. Use extracted key to make unauthenticated API calls
109. Download the app APK/IPA
110. Decompile using jadx/frida
111. Search for API key patterns
112. Use extracted key to make unauthenticated API calls
113. Download the app APK/IPA
114. Decompile using jadx/frida
115. Search for API key patterns
116. Use extracted key to make unauthenticated API calls
117. Download the app APK/IPA
118. Decompile using jadx/frida
119. Search for API key patterns
120. Use extracted key to make unauthenticated API calls
121. Download the app APK/IPA
122. Decompile using jadx/frida
123. Search for API key patterns
124. Use extracted key to make unauthenticated API calls
125. Download the app APK/IPA
126. Decompile using jadx/frida
127. Search for API key patterns
128. Use extracted key to make unauthenticated API callsa

Medium Risk Findings

MEDIUM **1 finding** Avg CVSS: 5.9

Medium-severity findings require non-trivial preconditions but materially weaken security posture. Remediate within 90 days.

ID	Title	CVSS	Page
F-P2-001	Broken object level authorization on payment endpoint	5.9	14

FINDING **F-P2-001**

Broken object level authorization on payment endpoint

SEVERITY	MEDIUM
CVSS V3.1	**5.9** AV:P/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:H
CWE	CWE-639
OWASP	A01:2021
AFFECTED	api.noriga.internal/v2/payments

Description

Payment API endpoints lack proper authorization checks at the object level. Authenticated users can access and modify payment records belonging to other accounts.

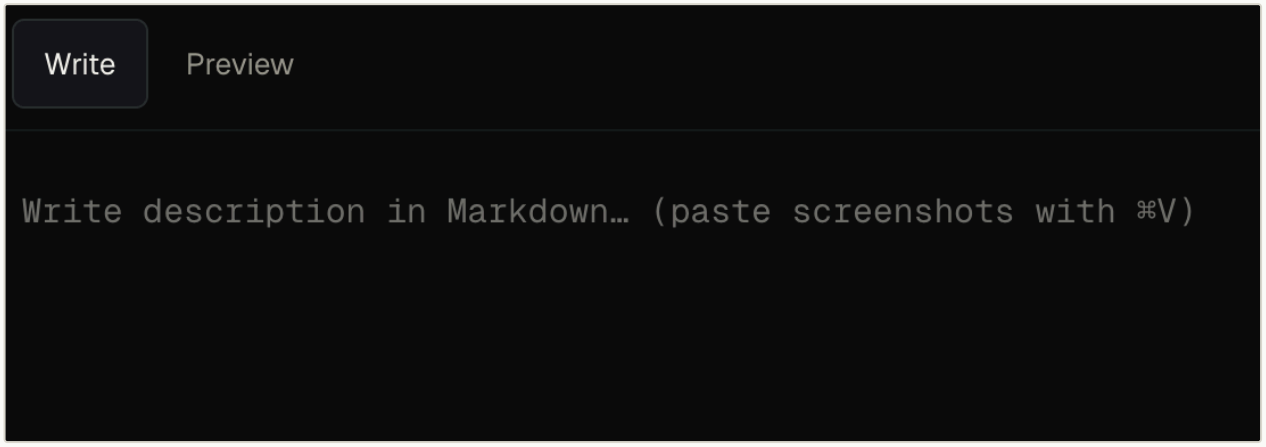

Figure. Screenshot 1

```javascript
var test = "Okay";
console.log(test);
```

Impact

- Unauthorized access to financial records. Potential fund manipulation across all customer accounts.
- No Test

Recommendations

Implement BOLA/IDOR protection by always verifying that the authenticated user owns the requested resource.

Technical Details

```javascript
   ...(repro ? [{
      key: 'repro',
```

FINDING **F-P2-001** *CONTINUED*

```
        node: (
          <div className="rpt-fsec">
            <div className="rpt-fsec-hd">Technical Details</div>
            <div className="rpt-fsec-rule" />
            <p className="rpt-p" style={{ whiteSpace: 'pre-wrap', fontFamily: 'var(--mono)',
  fontSize: '9pt', background: 'var(--ink5)', padding: '8pt', borderRadius: '4pt', border:
  '1pt solid var(--ink10)' }}>{repro}</p>
          </div>
        ),
```

Figure. *Screenshot 1*

Appendix A — CVSS Severity Ratings

Severity is assigned per finding using the *Common Vulnerability Scoring System v3.1*. Each finding records both the numerical score and vector string, allowing re-derivation against custom environmental context.

Severity	CVSS Range	Definition
CRITICAL	9.0 – 10.0	Direct, immediate threat. Exploitable by an unauthenticated attacker with low effort. **Remediate immediately.**
HIGH	7.0 – 8.9	Significant threat to sensitive systems or data, or a meaningful component of an attack chain. **Remediate within 30 days.**
MEDIUM	4.0 – 6.9	Threat requiring non-trivial preconditions but materially weakening posture. **Remediate within 90 days.**
LOW	0.1 – 3.9	Minor exposure with limited impact in isolation. **Address as part of regular hygiene within 180 days.**
INFORMATIONAL	0.0	Observation worth noting for hardening or hygiene. No immediate risk.

> **Note —** CVSS scores reflect the *base score* only. Temporal and environmental modifiers are noted where relevant. Clients are encouraged to apply their own environmental vector to adjust scores to their specific context.

Appendix B — Tools and Techniques

The following tools and techniques were employed during the assessment. All tools were used in accordance with the agreed rules of engagement.

Tool / Technique	Category	Purpose
`Burp Suite Professional`	Web Proxy	HTTP interception, active scanning, session analysis
`Nuclei`	Scanner	Template-based vulnerability detection across web surfaces
`Nmap / Naabu`	Network Scanner	Port discovery, service version detection, OS fingerprinting
`ffuf / Feroxbuster`	Fuzzing	Directory enumeration, parameter fuzzing, wordlist attacks
`Semgrep`	SAST	Static analysis for common vulnerability patterns in source code
`SQLMap`	Exploitation	Automated SQL injection detection and exploitation
`Amass / Subfinder`	OSINT / Recon	Subdomain enumeration and asset discovery
`JWT Tool`	Token Analysis	JWT signature algorithm confusion and secret brute-forcing
`Hashcat`	Password Cracking	Offline hash cracking with GPU acceleration
`Manual Testing`	Methodology	Logic flaws, business process abuse, authentication bypass

Appendix C — Glossary

Key terms used throughout this report are defined below.

Term	Definition
Attack Surface	The set of all points at which an attacker can enter or extract data from a system.
Authentication Bypass	A vulnerability allowing access to protected resources without valid credentials.
CVSS	Common Vulnerability Scoring System — a standardised framework for rating vulnerability severity.
CWE	Common Weakness Enumeration — a community-developed list of software and hardware weakness types.
IDOR	Insecure Direct Object Reference — direct access to objects (records, files) using user-controlled input.
Lateral Movement	Techniques used by attackers to progressively move through a network after initial compromise.
OWASP	Open Worldwide Application Security Project — provides freely available security guidance and tools.
Penetration Test	Authorised simulated cyberattack against a computer system to evaluate security weaknesses.
PoC	Proof of Concept — a demonstration that a vulnerability exists and can be exploited.
PTES	Penetration Testing Execution Standard — a common methodology framework for penetration tests.
Remediation	The process of fixing or mitigating a discovered security vulnerability.
SQL Injection	Insertion of malicious SQL code into a query via user-supplied input fields.
STRIDE	A threat modelling framework covering Spoofing, Tampering, Repudiation, Info Disclosure, DoS, Elevation.
XSS	Cross-Site Scripting — injection of malicious scripts into web pages viewed by other users.